

Lecture 6: SQL (Part 4)

Database Modification Statements

Instructor: Nikos Tziavelis

Reference:

A First Course in Database Systems, 3rd edition, Chapter 6.5

Slides credit:
Sheldon Finkelstein

Database Modification Statements

- SQL statements for:
 - *Inserting* some tuples into a relation
 - *Deleting* some tuples from a relation
 - *Updating* values of some columns of some existing tuples
- INSERT, DELETE, and UPDATE are referred to as *modification* operations.
 - They are Data Manipulation Language (DML) statements, as is SELECT.
- Modification operations change the *state* of the database.
 - They do not return a collection of rows or other values.
 - They may return errors/error codes.

Insert Statement with Values

```
INSERT INTO R(A1, ..., An)  
VALUES (v1, ..., vn);
```

- A tuple (v₁, ..., v_n) is inserted into the relation R, where attribute A_i = v_i and default values (perhaps NULL) are entered for all missing attributes.

```
INSERT INTO StarsIn(movieTitle, movieYear, starName)  
VALUES ('The Maltese Falcon', 1942, 'Sydney Greenstreet');
```

- The tuple ('The Maltese Falcon', 1942, 'Sydney Greenstreet') will be added to the relation StarsIn.

```
INSERT INTO StarsIn  
VALUES ('The Maltese Falcon', 1942, 'Sydney Greenstreet');
```

INSERT, DEFAULT and NULL

```
CREATE TABLE MovieStar (  
    starName      CHAR(30),  
    address       VARCHAR(255) DEFAULT 'Hollywood',  
    gender        CHAR(1) NOT NULL,  
    birthdate     DATE  
);
```

1. If a new row is inserted into MovieStar, and a value is specified for an attribute, then that attribute will receive that value.
2. If a new row is inserted into MovieStar, and no value is specified for an attribute, and that attribute has a DEFAULT (e.g., for address), then the value for that attribute will be that DEFAULT, which is 'Hollywood' for address.
3. If a new row is inserted into MovieStar, and no value is entered for an attribute, and the value of that attribute does not have a DEFAULT, and the value of that attribute is allowed to be NULL (e.g., for birthdate), then the value for the attribute will be NULL.
4. If a new row is inserted into MovieStar, and no value is entered for an attribute, and the value of the attribute does not have a DEFAULT, and the value is not allowed to be NULL (e.g., for gender), then an error will be raised.

INSERT Statement with Subquery

Movies(MovieTitle, MovieYear, length, genre, studioName, producerC#)

Studio(studioName, address, presC#)

(Temporarily, assume that there isn't Referential Integrity for studioName.)

```
INSERT INTO Studio(studioName)
  SELECT DISTINCT m.studioName
  FROM Movies m
  WHERE m.studioName NOT IN
    (SELECT st.studioName
     FROM Studio st);
```

Why DISTINCT?

- Add to the relation Studio all the studio names that appear in the studioName column of Movies but do not already occur in the studio names in the Studio relation.

INSERT Statement with Subquery and Constants

Movies(movieTitle, movieYear, length, genre, studioName, producerC#)
Disney2018Movies(movieTitle, length)

```
INSERT INTO Movies(movieTitle, movieYear, length, studioName)
  SELECT dm.movieTitle, 2018, dm.length, 'Disney'
  FROM Disney2018Movies dm;
```

Puts the Disney2018Movies into the Movies table, setting movieYear and studioName values using constants.

What will be the genre and producerC# values for the inserted movies?

DELETE Statement

```
DELETE FROM R  
  WHERE <condition>;
```

```
DELETE FROM StarsIn  
  WHERE movieTitle = 'The Maltese Falcon'  
    AND movieYear = 1942  
    AND starName = 'Sydney Greenstreet';
```

- The tuple ('The Maltese Falcon', 1942, 'Sydney Greenstreet') will be deleted from the relation StarsIn.
- What if we wanted to delete all tuples from StarsIn where the name of the star was Sydney Greenstreet?

More DELETE Examples

```
DELETE FROM MovieExec  
WHERE netWorth < 10000000;
```

- Deletes all movie executives whose net worth is less than 10 million dollars.

```
DELETE FROM MovieExec  
WHERE cert# IN  
  (SELECT m.producerC#  
   FROM Movies m, StarsIn s  
   WHERE m.movieTitle = s.movieTitle AND m.movieYear = s.movieYear  
        AND s.starName = 'Sydney Greenstreet');
```

- Deletes all movie executives who produced movies starring Sydney Greenstreet

DELETE: Careful

What does:

```
DELETE FROM MovieExec;
```

without a WHERE clause do?

Answer: Deletes all the tuples from MovieExec!!!

UPDATE Statement

UPDATE R

SET <new-value-assignments>
WHERE <condition>;

- <new-value-assignment> :-
 <attribute> = <expression>, ..., <attribute> = <expression>

UPDATE Employees

SET salary = 85000, dept = 'SALES'
WHERE Ssnum = '123456789';

UPDATE Employees

SET salary = 25000
WHERE salary IS NULL;

UPDATE Employees

SET salary = salary * 1.1
WHERE salary > 100000;

UPDATE with Subquery

UPDATE R

SET <new-value-assignments>

WHERE <condition>;

- <new-value-assignment>:-
 <attribute> = <expression>, ..., <attribute> = <expression>

UPDATE MovieExec

SET execName = 'Pres. ' || execName

WHERE cert# IN (SELECT presC# FROM Studio);

- 2nd line: concatenates the string 'Pres. ' with execName.

Another UPDATE with Subquery (which might help you do Lab3)

- The **UPDATE** clause specifies the table being updated (just one table).
- A SQL UPDATE statement can also have a **FROM** clause.
- The UPDATE statement's **WHERE** clause can refer to both the updated table and the table(s) in the FROM clause.
- The UPDATE statement's **SET** clause can also refer to the updated table and the table(s) in the FROM clause, but attributes that are SET in the updated table must appear without the tuple variable.

Tickets(airlineID, flightNum, ticketID, custID, seatNum, paid)
NewTickets(airlineID, flightNum, ticketID, custID, seatNum);

```
UPDATE Tickets t
SET seatNum = newt.seatNum, paid = FALSE
FROM NewTickets newt
WHERE t.ticketID = newt.ticketID
      AND t.custID = newt.custID
      AND t.airlineID = newt.airlineID
      AND t.flightNum = newt.flightNum;
```

You may use tuple variables in the UPDATE, FROM and WHERE clauses of an UPDATE statement ... but attributes that are SET in the updated table must appear without the tuple variable.

Semantics of Modifications

- Database modification statements are completely evaluated on the old state of the database, producing a new state of the database.
 - What does this statement do? Is it deterministic or not?

```
UPDATE MovieExec e
  SET netWorth = 6M
  WHERE NOT EXISTS (SELECT * FROM MovieExec e2
                    WHERE e2.Networth = 6M);
```

Should write 6000000. 6M is clearer (but it's not legal SQL).

MovieExec

execName	address	cert#	netWorth
S. Spielberg	X	38120	3M
G. Lucas	Y	43918	4M
W. Disney	Z	65271	5M